Modelado de datos en bases de datos (NoSQL)

Introducción

El modelado en MongoDB consiste en establecer cómo vamos a **identificar** y **representar**:

→ las entidades del dominio,
→ sus atributos,
→ relaciones.

MongoDB no impone esquemas fijos y su diseño debe centrarse en cómo los datos serán consultados y actualizados, optimizando el rendimiento y la escalabilidad.

Entidades → Son parte de la información y van a tener significado por sí mismas.

- Son **únicas** e **independientes** de las demás.
- Representan objetos, personas o conceptos dentro de nuestra app.
- Se agrupan en colecciones.

Atributos → Describen una entidad u objeto, además son los pares clave-valor que vemos entre documentos (siguen la regla "los datos a los que se accede conjuntamente deben almacenarse juntos")

Diseño de modelos en NoSQL

Su diseño busca equilibrar la redundancia con el rendimiento. Alguna de sus decisiones clave incluyen la utilización de:

Embebido de documentos cuando:

→ Representamos estructuras jerárquicas o relaciones estrechas entre entidades
→ Los datos se leen/escriben juntos.
→ La data embebida no cambia habitualmente, o lo hace a través del padre.
→ La data embebida es sólo una parte de una entidad más grande, dejando afuera atributos no relevantes, reduciendo la redundancia.

Referencias entre documentos cuando:

→ La relación es más débil o de gran tamaño, permitiendo mantener la independencia y escalabilidad de los datos.

Generalización y especialización

MongoDB permite modelar jerarquías de tipo "**especialización/generalización**" (herencia) mediante diferentes enfoques:

- **Un sólo documento por jerarquía**: todos los subtipos se guardan en la misma colección, diferenciándose por un campo tipo ("type": "Profesor", "Type": "Alumno").
- **Colecciones separadas por subtipo**: cada entidad hija se almacena en su propia colección, heredando atributos comunes del documento padre.
- **Modelo híbrido**: combinación de ambos métodos, manteniendo atributos generales en una colección base y los específicos embebidos o referenciados según el caso.

Su elección depende del → Volumen de datos → Frecuencia de acceso → Complejidad de las consultas.

Relaciones en distintos contextos. Estructuras primitivas, documentos embebidos, referencias, colecciones asociativas.

Introducción

Fundamental entender cómo modelar relaciones **uno a uno**, **uno a muchos** y **muchos a muchos**, en diferentes contextos, usando estructuras primitivas, documentos embebidos, referencias o colecciones asociativas.

Estructuras primitivas

En MongoDB se usan arrays y subdocumentos como estructuras de datos primitivas. Estas estructuras son fundamentales para modelar relaciones embebidas o listas de elementos relacionados.

Embebido vs referencia y patrones híbridos:

→ El **embebido** reduce la necesidad de consultas múltiples, pero puede hacer que el documento crezca demasiado.

→ La **referencia** mejora modularidad, pero requiere más consultas.

(También se puede usar un **patrón híbrido**: embebemos algunos campos esenciales para consultas rápidas y referenciamos detalles menos usados).

Relaciones → uno a uno
Existe cuando una entidad está relacionada solamente a una entidad.
Ejemplo: cada usuario tiene exactamente un perfil de detalle.

Embebido (si siempre consultas al usuario con su perfil):

```
{
  _id: ObjectId("U1"),
  nombre: "María",
  email: "maria@example.com",
  perfil: {
    fecha_nacimiento: "1990-05-10",
    direccion: "Córdoba 123"
  }
```

Referencia (si el perfil contiene muchos datos o es independiente):

```
{
  _id: ObjectId("U1"),
  nombre: "María",
  email: "maria@example.com",
  perfil_id: ObjectId("P1")
}

{
  _id: ObjectId("P1"),
  fecha_nacimiento: "1990-05-10",
  direccion: "Córdoba 123",
  teléfono:  “123123123” ,
}
```

Relaciones → uno a muchos
Existe cuando una entidad está relacionada a múltiples instancias de otra entidad.
Ejemplo: un curso tiene varios estudiantes.

Embebido

```
{
  _id: ObjectId("Curso1"),
  nombre: "Base de Datos II",
  estudiantes: [
    { estudiante_id: ObjectId("E1"), nombre: "Valentina" },
    { estudiante_id: ObjectId("E2"), nombre: "Juan" }
  ]
}
```

Referencia

```
{
  _id: ObjectId("E1"),
  nombre: "Valentina",
  curso_id: ObjectId("Curso1")
}
```

Relaciones → muchos a muchos
Existe cuando muchas instancias de una entidad se relacionan con muchas instancias de otra entidad. En MongoDB, para resolver este escenario se presentan dos estrategias alternativas que deben elegirse según la necesidad del sistema.
Ejemplo: Estudiantes y cursos tienen relación mutua: un estudiante puede estar en muchos cursos, un curso tiene muchos estudiantes.

Estrategia 1, Colección asociativa: Es el enfoque recomendado para mantener la independencia y es ideal cuando la relación posee atributos propios.

Inscripciones:

```
{
  _id: ObjectId("I1"),
  estudiante_id: ObjectId("E1"),
  curso_id: ObjectId("Curso1"),
  fecha_inscripcion: "2025-10-05"
}
```

Estrategia 2, Arreglos de referencias: Consiste en incluir listas de IDs dentro de los documentos principales para agilizar consultas frecuentes sin necesidad de colecciones extra.

Estudiantes

```
{
  _id: ObjectId("E2"),
  nombre: "Juan",
  curso_ids: [ ObjectId("Curso1"), ObjectId("Curso2") ]
}
```

Cursos

```
{
  _id: ObjectId("Curso1"),
  nombre: "Base de Datos II",
  estudiante_ids: [ ObjectId("E1"), ObjectId("E2") ]
}
```

Las colecciones asociativas y los arreglos de referencias deben considerarse alternativas de modelado, no estructuras que necesariamente deban coexistir.

Si se decide combinarlas como estrategia de optimización (patrón híbrido), se debe tener control extremo de la redundancia, definiendo claramente y utilizando mecanismos de consistencia para evitar inconsistencias.

La coexistencia sin un diseño explícito puede generar duplicación innecesaria y problemas de integridad.

Bases de datos para aplicaciones móviles

Bases de datos móviles/edge con MongoDB

MongoDB ofrece soluciones donde los datos necesitan estar disponibles sin conexión constante, y luego sincronizarse con la nube cuando sea posible, a través de servicios como **MongoDB Atlas**, por ejemplo, manteniendo un equilibrio entre **rendimiento, disponibilidad y constancia**.

Su clasificación puede abarcar:

- BD locales en dispositivos con sincronización eventual.
- BD cloud + edge, donde parte de los datos que se encuentran localmente.
- Replicación y sincronización entre dispositivos y servidores.

TIPO DE BASE DE DATOS	DESCRIPCIÓN	EJEMPLO
Local (on-device)	Los datos se almacenan en el dispositivo móvil. Funcionan sin conexión y se sincronizan con la nube cuando hay red.	Realm (MongoDB Atlas Device SDK), SQLite, Core Data
Remota (cloud-based)	Los datos se almacenan en servidores o servicios en la nube y se accede mediante API o SDK.	MongoDB Atlas, Firebase, Couchbase Cloud
Híbrida/sincronizada (Edge/Sync)	Combina ambos almacenamientos. Guarda datos en el dispositivo temporalmente y los sincroniza mediante un proceso de *sync* o *replicación eventual*.	MongoDB Atlas Devide Sync, Couchbase Mobile

Propiedades de consistencia en MongoDB

MongoDB **NO** define niveles de consistencia de la misma forma que otras BD, pero permite control con:

→ **Write concern**: cuantos nodos deben confirmar una escritura antes de considerarla exitosa.
→ **Read concern**: qué tan "reciente" debe ser la lectura.
→ **Transacciones**: permiten agrupar múltiples operaciones.
→ **Consistencia y sesión actual**: MongoDB soporta sesiones con consistencia causal, lo que garantiza que lecturas y escrituras respeten el orden lógico de las operaciones en esa sesión.

Además, cuando duplicamos datos entre varias colecciones, necesitamos mecanismos para mantener la consistencia.

Ejemplos de desarrollo móvil con sincronización

Una app de inventario en un en un dispositivo móvil debe funcionar sin conexión:

El cliente → tiene una base local donde se registra movimientos de stock → luego se sincroniza con el servidor central.

- En el celular, cada **movimiento** se guarda localmente con campos como → { id_local, producto_id, cantidad, fecha }.
- Cuando el dispositivo recupera conexión, sincroniza estos movimientos con la **colección** de movimientos en el servidor.
- Usamos transacciones o sesiones con *write/read concern* apropiado para asegurar que la sincronización no genere conflictos ni inconsistencias.

Instancias y decisiones de diseño

A la hora de tomar decisiones tenemos que tener en cuenta:

- ¿Aceptamos consistencia eventual? → retrasos en la sincronización
- ¿Se necesita consistencia fuerte? → las escrituras deben reflejarse rápidamente.
- Las decisiones de consistencia afectan el rendimiento y disponibilidad.
- En entornos móviles, la tolerancia a desconexión es crítica → Es de suma importancia poder operar el sistema sin contar en todo momento con una conexión a internet. (sincronización)
- Manejar conflictos es parte del diseño → se pueden implementar estrategias como por ejemplo “último que escribe”, entre otras.